

I am working on a machine learning project in VS Code using...

I am working on a machine learning project in VS Code using Codex. I want you to act as my AI-assisted coding partner throughout the project.

For this first step, DO NOT start writing the full solution or modifying files yet. First inspect the files available in my current VS Code workspace, including the learner guide notebook, any starter notebook, and the dataset `loan_applications.csv`. Then create a clear step-by-step implementation plan that follows the learner guide and the grading requirements described below.

PROJECT BUSINESS CONTEXT

Beacon Community Capital is a Treasury-certified Community Development Financial Institution based in Columbus, Ohio. It provides small personal and small-business loans to borrowers who may be underserved by traditional banks, including first-time borrowers, immigrants, and local business owners rebuilding after financial setbacks.

Beacon must balance two priorities:

1. Manage credit risk and avoid excessive defaults so that the organization remains financially sustainable.
2. Continue serving borrowers who may not qualify under traditional lending criteria.

Beacon currently has five loan officers who manually review every application. Applications have increased from about 500 per month to about 1,800 per month while staffing has remained unchanged.

Average decision time is now three to four business days.

An internal review also found that approximately one in five historical decisions would have been different when independently reviewed by another officer. Decision reasoning is also not consistently documented.

Beacon previously considered a simple rule:

- Approve applicants with FICO score ≥ 660
- Reject applicants with FICO score < 660

However, this was too simplistic because applicants with lower FICO scores may still have other positive characteristics such as stable income, employment, or savings.

PROJECT OBJECTIVE

Build a reproducible machine learning workflow using Python in a Jupyter notebook in VS Code.

The model must be a logistic regression classifier.

The model should predict Beacon's HISTORICAL LOAN OFFICER APPROVAL DECISION. It is NOT predicting whether a borrower will repay a loan or default.

The target variable is `approved`.

The model should produce predicted approval probabilities between 0 and 1.

The following business thresholds must then be applied:

- Probability ≥ 0.80 : likely approval, eligible for an expedited approval workflow
- Probability ≥ 0.20 and < 0.80 : borderline case, mandatory loan officer review
- Probability < 0.20 : likely rejection, mandatory human officer review

These thresholds are business-defined. They are not thresholds

learned by logistic regression.

The model must NOT automatically reject applicants. Human loan officers remain responsible for final lending decisions.

DATASET

The dataset is `loan_applications.csv`

Each row represents one historical loan application.

Columns:

- `applicant_id`: unique application identifier
- `fico_score`: applicant credit score
- `annual_income`: yearly income in dollars
- `loan_amount`: amount requested in dollars
- `loan_term_months`: repayment length in months
- `employment_status`: employment type
- `years_employed`: years in current employment
- `savings_balance`: savings balance in dollars
- `approved`: historical decision made by Beacon's credit team

Target:

- `approved` = 1 means approved
- `approved` = 0 means rejected

IMPORTANT MODEL RULES

`approved` must never be included as an input feature.

`applicant_id` is only an identifier and must not be used as a model feature.

If we create an `approved_flag` column during data preparation, it must also be excluded from X when training the model.

PROJECT STAGES

The project has four broad stages:

1. Load and clean the data
2. Explore historical approvals and application characteristics
3. Train a logistic regression classification model
4. Evaluate and interpret model performance

GRADING REQUIREMENTS

Please design the plan so that the final notebook explicitly covers every item below.

A. DATA INSPECTION AND CLEANING

1. Load `loan_applications.csv`.
2. Display:
 - dataset shape
 - first rows
 - column data types
 - missing/null values
 - summary statistics using `describe()`
3. Create a bar chart showing missing values.
4. Allow me to write five observations about the RAW dataset based on the displayed information and charts.
5. Correctly clean:
 - money columns stored as text by converting them to numeric values
 - duplicate rows
 - impossible or invalid values
 - blank/missing FICO scores
6. Document each cleaning decision and avoid silently dropping data without explanation.
7. Preserve the cleaned dataset in a clearly named `DataFrame` and, if appropriate according to the learner guide, save a cleaned CSV version.

Before deciding how to handle missing FICO scores, invalid values, duplicates, or other data-quality issues, inspect the actual dataset and explain the proposed approach. Do not assume what problems exist before examining the data.

B. EXPLORATORY DATA ANALYSIS

1. Approved versus rejected counts and a bar chart.
2. FICO score distribution histogram.
3. Approval rate by employment type and a bar chart.
4. FICO score by approval decision box plot.
5. Correlation heatmap showing the relationships between the six relevant numeric input variables and approval outcome.
6. Five EDA findings that can be directly supported by the actual charts and outputs.

Do not invent findings before running the analysis. Generate the charts first, then help me interpret what is actually visible.

C. FAIRNESS REASONING

1. Could using income, savings, and employment information still result in unfairness toward some groups? Answer yes or no and provide a meaningful one-sentence reason.
2. Why should applications in the borderline probability band be reviewed by a human instead of being automatically decided?
3. What is the risk of training a model to reproduce historical loan officer decisions?

The discussion must recognize that historical decisions may contain human bias and that the model predicts past officer behavior rather than true borrower creditworthiness or future repayment.

D. MODEL PREPARATION AND TRAINING

1. Convert or verify the target approval variable as a clean binary 0/1 variable.
2. Construct X and y correctly.
3. Exclude:
 - `applicant_id`
 - `approved`
 - `approved_flag`, if it exists
4. Correctly handle the categorical `employment_status` feature for logistic regression.
5. Split data into training and test sets.
6. Use a fixed `random_state` for reproducibility.
7. Use a stratified split based on y.
8. Train a logistic regression classifier on the TRAINING DATA

ONLY.

9. Use preprocessing that avoids data leakage.
10. Use scaling where appropriate for numeric features so logistic regression coefficients can also be compared fairly.
11. Keep preprocessing and modelling reproducible, preferably using an appropriate scikit-learn Pipeline and ColumnTransformer if this is consistent with the learner guide.

E. FEATURE INTERPRETATION

Create:

1. A chart showing the logistic regression feature coefficients/ weights.
2. A fairly comparable feature-importance or coefficient chart using appropriately scaled numeric variables.

Clearly explain that logistic regression coefficients represent associations with the HISTORICAL approval decision and should not automatically be interpreted as causal relationships.

F. MODEL EVALUATION

Evaluate the model on the TEST SET, not the training set.

Use appropriate classification evaluation metrics. At minimum, plan for metrics that meaningfully describe classification performance, and follow any additional metrics required by the learner guide.

Also generate predicted approval probabilities using `predict_proba()`.

For every test application, apply the project thresholds:

- >0.80 , expedited/ approval
- ≥ 0.20 and <0.80 , send to human for review
- < 0.20 : Likely rejection / human review required

Show the number of test applications falling into each confidence band.

Do not call the low-probability category an automatic rejection in the

final interpretation because the business description explicitly requires human review before any final rejection.

G. FINAL INTERPRETATION

1. What the model's test results actually showed.
2. What those results mean in the Beacon business context.
3. Why considering several application characteristics together, such as FICO, income, savings, employment, requested loan amount, and employment history, can provide a better approximation of historical officer decisions than using one strict FICO cutoff.
4. Model strengths.
5. Model limitations.
6. Appropriate use of the model.
7. Why the model must remain a decision-support / first-pass sorting tool rather than a replacement for loan officers.

Conclusions must be based on the actual model results rather than being invented in advance.

H. README AND SUBMISSION

At a later stage I will ask you to generate a `README.md` containing:

- project overview
- business problem
- project objective
- dataset description
- four broad project stages
- setup instructions
- libraries/dependencies
- how to run the notebook
- summary of cleaning performed
- modelling approach
- key results
- limitations
- appropriate use of the model

For this response, provide only:

1. A brief summary of your understanding of the business problem.
2. A summary of what you found in the learner guide notebook and project files.
3. A numbered implementation plan mapped to the grading requirements.
4. A proposed notebook structure with Markdown section headings.
5. The expected Python libraries.
6. Any issues or decisions that should wait until we inspect the dataset.

Do not start implementing the project yet.

16 previous messages

Metric	Value
Tokens quota	92,457/100,000,000 used, 99,907,543 remaining
Resets in	6d 23h 57m 55s
Total consumed	92,457 tokens, \$0.082845 spent
Request tokens	14,233 in / 353 out (301 reasoning)

Metric	Value
Tokens quota	108,072/100,000,000 used, 99,891,928 remaining
Resets in	6d 23h 57m 44s
Total consumed	108,072 tokens, \$0.095500 spent
Request tokens	14,953 in / 662 out (257 reasoning)

Metric	Value
Tokens quota	124,677/100,000,000 used, 99,875,323 remaining
Resets in	6d 23h 57m 38s
Total consumed	124,677 tokens, \$0.107286 spent
Request tokens	16,106 in / 499 out (133 reasoning)

Metric	Value
Tokens quota	142,814/100,000,000 used, 99,857,186 remaining
Resets in	6d 23h 57m 26s
Total consumed	142,814 tokens, \$0.119902 spent
Request tokens	17,641 in / 496 out (127 reasoning)

Metric	Value
Tokens quota	162,163/100,000,000 used, 99,837,837 remaining
Resets in	6d 23h 57m 19s
Total consumed	162,163 tokens, \$0.131135 spent
Request tokens	18,925 in / 424 out (188 reasoning)

Metric	Value
Tokens quota	182,071/100,000,000 used, 99,817,929 remaining
Resets in	6d 23h 57m 14s

Total consumed	182,071 tokens, \$0.139451 spent
Request tokens	19,654 in / 254 out (83 reasoning)

Metric	Value
Tokens quota	202,578/100,000,000 used, 99,797,422 remaining
Resets in	6d 23h 57m 6s
Total consumed	202,578 tokens, \$0.149816 spent
Request tokens	20,056 in / 451 out (110 reasoning)

Metric	Value
Tokens quota	223,660/100,000,000 used, 99,776,340 remaining
Resets in	6d 23h 56m 53s
Total consumed	223,660 tokens, \$0.160197 spent
Request tokens	20,679 in / 403 out (179 reasoning)

Metric	Value
Tokens quota	245,577/100,000,000 used, 99,754,423 remaining
Resets in	6d 23h 56m 48s
Total consumed	245,577 tokens, \$0.169519 spent
Request tokens	21,621 in / 296 out (146 reasoning)

Metric	Value
Tokens quota	267,916/100,000,000 used, 99,732,084 remaining
Resets in	6d 23h 56m 43s
Total consumed	267,916 tokens, \$0.177219 spent
Request tokens	22,104 in / 235 out (63 reasoning)

Metric	Value
Tokens quota	291,163/100,000,000 used, 99,708,837 remaining
Resets in	6d 23h 56m 38s
Total consumed	291,163 tokens, \$0.186582 spent
Request tokens	22,973 in / 274 out (116 reasoning)

Metric	Value
Tokens quota	315,113/100,000,000 used, 99,684,887 remaining
Resets in	6d 23h 56m 34s
Total consumed	315,113 tokens, \$0.195115 spent
Request tokens	23,729 in / 221 out (50 reasoning)

Metric	Value
Tokens quota	340,085/100,000,000 used, 99,659,915 remaining
Resets in	6d 23h 56m 26s

Total consumed	340,085 tokens, \$0.206470 spent
Request tokens	24,568 in / 404 out (348 reasoning)

Metric	Value
Tokens quota	366,241/100,000,000 used, 99,633,759 remaining
Resets in	6d 23h 56m 17s
Total consumed	366,241 tokens, \$0.218657 spent
Request tokens	25,705 in / 451 out (209 reasoning)

Metric	Value
Tokens quota	393,973/100,000,000 used, 99,606,027 remaining
Resets in	6d 23h 55m 54s
Total consumed	393,973 tokens, \$0.243247 spent
Request tokens	26,362 in / 1,370 out (1,258 reasoning)

1. Business problem (my understanding)

- Beacon needs a fast, consistent first-pass triage of loan applications that reproduces historical loan officer approval decisions (not default risk), so officers can focus time on borderline cases while keeping humans responsible for final approvals/rejections.

2. What's in your workspace + learner guide summary

- Files present: `Learner_Guide_Notebook.ipynb`, `loan_applications.csv` (no separate starter notebook detected).

- Learner guide structure: 4 stages (cleaning → EDA/business insights → logistic regression training → evaluation) plus written fairness questions and conclusion prompts; it includes “expected result” checkpoints (e.g., duplicates removed, final clean row count, expected accuracy ranges).
- Dataset quick facts (raw): 232 rows × 9 columns; `fico_score` has 10 missing; `annual_income`, `loan_amount`, `savings_balance` are mixed text formats (some rows include \$ and commas); `approved` is text (“Approved/Rejected”); there are 12 exact duplicate rows; there are clearly impossible values present (e.g., `fico_score` up to 900, `loan_term_months` up to 999, `years_employed` up to 150), matching the guide’s “remove impossible values” step.

3. Numbered implementation plan (mapped to grading requirements) A. Data inspection & cleaning

1. (A1–A2) Load `loan_applications.csv` into `df_raw`; display shape, head, dtypes, null counts, `describe()`.
2. (A3) Create a missing-values bar chart (counts per column).
3. (A4) Add a Markdown cell for you to write 5 raw-data observations (based only on the outputs/charts).
4. (A5–A6) Cleaning, explicitly documented with before/after row counts and rationale:
 - Fix money-as-text: strip \$/, then convert `annual_income`, `loan_amount`, `savings_balance` to numeric (show dtype confirmation).
 - Remove exact duplicates (guide expects 12 removed).
 - Identify and remove rows with “present but impossible” values (define rules after inspecting `describe()/min/max`; guide expects 8 removed).
 - Handle missing FICO: fill with median FICO (guide expects 10 filled; 0 missing afterward).
5. (A7) Preserve a clearly named cleaned DataFrame (e.g., `df_clean`); optionally export a cleaned CSV only if it aligns with your course expectations (the learner guide doesn’t require a save).

B. Exploratory data analysis 6. (B1) Approved vs rejected counts + bar chart. 7. (B2) FICO histogram. 8. (B3) Approval rate by

`employment_status` + bar chart. 9. (B4) Box plot of FICO by approval decision. 10. (B5) Correlation heatmap for the six numeric inputs + approval outcome (numeric: `fico_score`, `annual_income`, `loan_amount`, `loan_term_months`, `years_employed`, `savings_balance`, plus the numeric approval target). 11. (B6) Add a Markdown cell for you to write 5 EDA findings supported by the charts (no invented findings).

C. Fairness reasoning 12. (C1–C3) Answer the 3 fairness questions in Markdown, explicitly noting the model reproduces historical officer behavior and can inherit bias; reinforce why the 0.20–0.80 band requires human review and why low probability is not an auto-reject.

D. Model preparation & training (logistic regression) 13. (D1) Create a clean binary target (e.g., `approved_num`: Approved→1, Rejected→0) and verify counts. 14. (D2–D3) Build X and y correctly; exclude `applicant_id`, `approved`, and any temporary flags/targets (e.g., `approved_num` / `approved_flag`) from features. 15. (D4) Handle `employment_status` for logistic regression:

- Follow the learner guide approach (create `is_self_employed` indicator) or, if your rubric prefers, one-hot encode; confirm the approach before fitting.
16. (D5–D8) Split train/test with `test_size=0.2`, `random_state=42`, `stratify=y`; train `LogisticRegression(max_iter=1000)` on training data only.
 17. (D9) Prevent leakage: any scaling/encoding fitted only on training data (Pipeline/ColumnTransformer if you choose; otherwise, very explicit “fit on train, transform train/test”).
 18. (D10–D11) Scaling: keep the main model aligned with the guide; train a separate scaled version (fit scaler on `X_train` only) specifically for fair coefficient comparison.

E. Feature interpretation 19. (E1) Plot main-model coefficients (feature → weight) with clear labeling. 20. (E2) Plot scaled-model coefficients for more comparable “importance”. 21. Add a brief explanation: coefficients are associations with historical approvals, not causal effects.

F. Model evaluation 22. (F) Evaluate on test set only:

- At minimum: accuracy (matches learner guide), plus (recommended for “appropriate metrics”): confusion matrix + precision/recall/F1 and/or ROC-AUC.

23. Generate approval probabilities with `predict_proba()[0, 1]`.

24. Apply business thresholds to test predictions:

- `>= 0.80`: likely approval → expedited path
- `0.20–0.79`: borderline → mandatory officer review
- `< 0.20`: likely rejection → mandatory officer review (no auto-reject language)

25. Show counts in each probability band (and optionally a bar chart).

G. Final interpretation 26. Write conclusions based on your actual test results:

- What metrics show; what this means for Beacon’s workflow; why multi-factor beats a hard FICO cutoff; strengths/limitations; appropriate use as decision support; why humans remain accountable.

H. README & submission (later) 27. When you ask, draft `README.md` covering the required items and summarizing what the notebook actually did/results it produced.

4. Proposed notebook structure (Markdown section headings)

- `# Business Overview`
- `# Objective and Constraints (What We Predict / What We Don’t)`
- `# Stage 1 – Data Inspection (Raw)`
- `# Stage 1 – Cleaning Decisions (Documented)`
- `# Stage 2 – Exploratory Data Analysis`
- `# Fairness Reasoning`
- `# Stage 3 – Model Preparation`
- `# Stage 3 – Logistic Regression Training`

- # Feature Interpretation (Coefficients)
- # Stage 4 – Evaluation and Baselines (FICO>=660, Approve-All)
- # Workflow Threshold Bands (0.80 / 0.20)
- # Final Interpretation (Business Context)
- # Appendix (Optional: Data Dictionary / Extra Checks)

5. Expected Python libraries

- Core: pandas, numpy
- Viz: matplotlib, seaborn
- sklearn: train_test_split, LogisticRegression, metrics (accuracy_score at minimum; likely also classification_report, confusion_matrix, roc_auc_score), preprocessing (StandardScaler; optionally OneHotEncoder, ColumnTransformer, Pipeline if used)

6. Issues/decisions to wait until deeper dataset inspection (in-notebook, before cleaning)

- Exact “impossible value” rules to apply (we’ve confirmed they exist; we’ll define the cutoffs explicitly after reviewing describe() and domain realism so we remove the intended ~8 rows).
- Whether to encode employment_status as a single is_self-employed flag (matches the learner guide) vs one-hot encoding (more general, may shift results).
- Whether to export a cleaned CSV (not required by the learner guide; depends on your course submission expectations).

Metric	Value
Tokens quota	423,893/100,000,000 used, 99,576,107 remaining
Resets in	6d 23h 55m 21s
Total consumed	423,893 tokens, \$0.278175 spent
Request tokens	27,887 in / 2,033 out (342 reasoning)

That sounds good. First step we will be working on is loading the data and checking the size of it. Please help me with the code/syntax to load the data into "df" and print the shape of it to see and understand how it looks.

1 previous message

Use this in your notebook:

```
import pandas as pd

df = pd.read_csv("loan_applications.csv")

print("df.shape:", df.shape)  # (rows, columns)
df.head()
```

Metric	Value
Tokens quota	450,082/100,000,000 used, 99,549,918 remaining
Resets in	6d 23h 54m 41s
Total consumed	450,082 tokens, \$0.302217 spent
Request tokens	26,141 in / 48 out (0 reasoning)

Now, lets look at the first 5 rows, to see what the table looks like and how the data is. Print the first 5 rows.

1 previous message

Run:

```
df.head(5)
```

(Or `print(df.head(5))` if you prefer plain text output.)

Metric	Value
--------	-------

Tokens quota	476,444/100,000,000 used, 99,523,556 remaining
Resets in	6d 23h 51m 28s
Total consumed	476,444 tokens, \$0.307628 spent
Request tokens	26,329 in / 33 out (0 reasoning)

The next step is to take a look at the columns and their data types along with that we will also check which ones have non-null values and missing values count.

1 previous message

Use these to inspect columns, dtypes, non-null counts, and missing values:

```
# Column names + non-null counts + dtypes
df.info()

# Just the dtypes in a compact view
df.dtypes

# Missing (null) counts per column
df.isna().sum()
```

Metric	Value
Tokens quota	503,016/100,000,000 used, 99,496,984 remaining
Resets in	6d 23h 49m 56s
Total consumed	503,016 tokens, \$0.313634 spent
Request tokens	26,504 in / 68 out (0 reasoning)

Let's also look at the summary statistics to understand the numerical and categorical columns data. Through this we can also understand any funny values.

1 previous message

Use `describe()` for both numeric-only and “all columns” views:

```
# Numeric summary stats (count/mean/std/min/percentiles)
df.describe()

# Include categorical columns too (counts/unique/top/frequency)
df.describe(include="all")
```

If you want transposed output (often easier to read in notebooks):

```
df.describe(include="all").T
```

Metric	Value
Tokens quota	529,839/100,000,000 used, 99,470,161 remaining
Resets in	6d 23h 48m 11s
Total consumed	529,839 tokens, \$0.319945 spent
Request tokens	26,733 in / 90 out (0 reasoning)

This gives a lot of context, especially that the three money columns are text data types, the fico score is more than 850 and other funny values that we will eliminate later.

Now, for a better picture, please give me the code to build a bar chart using `matplotlib` lib that shows missing values within all the columns. Please title the graph as "Missing Values by Column" and give names to the y and x-axis. Also rotate the x-axis column names for better readability.

1 previous message

```
import matplotlib.pyplot as plt

missing_counts = df.isna().sum()
```

```
plt.figure(figsize=(10, 5))
plt.bar(missing_counts.index, missing_counts.values)
plt.title("Missing Values by Column")
plt.xlabel("Columns")
plt.ylabel("Missing Value Count")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	556,955/100,000,000 used, 99,443,045 remaining
Resets in	6d 23h 44m 59s
Total consumed	556,955 tokens, \$0.326256 spent
Request tokens	27,035 in / 81 out (0 reasoning)

This shows us that only Fico score is a column with missing values. Great. Let us now fix the issues. I want to convert the money columns 'annual_income', 'loan_amount', 'savings_balance' to numeric. They have values stored as \$53,000, so please help me remove the \$ and the , and convert them. Once that is done also print the data types of the three money columns to double check the change.

1 previous message

```
money_cols = ["annual_income", "loan_amount", "savings_balance"]

for col in money_cols:
    df[col] = (
        df[col]
        .astype(str)
        .str.replace(r"[\$,]", "", regex=True) # remove $ and ,
        .replace("nan", pd.NA)                # keep nan as NA
    )
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

```
# Double-check dtypes
df[money_cols].dtypes
```

Metric	Value
Tokens quota	584,420/100,000,000 used, 99,415,580 remaining
Resets in	6d 23h 42m 9s
Total consumed	584,420 tokens, \$0.333216 spent
Request tokens	27,348 in / 117 out (0 reasoning)

Next, we need to remove duplicates. Since there are rows that have been repeating, we need to eliminate them to prevent bias. drop the duplicates only and reset the index. Once that is done please give the initial number of rows, the count of duplicates that were removed and the overall row count remaining.

1 previous message

```
initial_rows = df.shape[0]
duplicate_count = df.duplicated().sum()

df = df.drop_duplicates().reset_index(drop=True)

remaining_rows = df.shape[0]

print("Initial rows:", initial_rows)
print("Duplicates removed:", duplicate_count)
print("Remaining rows:", remaining_rows)
```

Metric	Value
Tokens quota	612,150/100,000,000 used, 99,387,850 remaining

Resets in	6d 23h 38m 51s
Total consumed	612,150 tokens, \$0.339446 spent
Request tokens	27,661 in / 69 out (0 reasoning)

Let's also remove any impossible values that could have possibly shown up due to data entry errors. We do not want to remove any blank rows. check the initial row count and then remove rows based on the following conditions:

**- fico_score > 850 (and
fico_score is not NaN)**

- annual_income < 0

- loan_amount <= 0

- loan_term_months > 84

- years_employed > 60

- savings_balance < 0

reset the index and give the count of rows removed and the remaining rows.

1 previous message

```
initial_rows = df.shape[0]

impossible_mask = (
    ((df["fico_score"].notna()) & (df["fico_score"] > 8
    (df["annual_income"] < 0) |
    (df["loan_amount"] <= 0) |
    (df["loan_term_months"] > 84) |
    (df["years_employed"] > 60) |
    (df["savings_balance"] < 0)
)

removed_rows = int(impossible_mask.sum())

df = df.loc[~impossible_mask].reset_index(drop=True)

remaining_rows = df.shape[0]

print("Initial rows:", initial_rows)
print("Rows removed (impossible values):", removed_rows)
print("Remaining rows:", remaining_rows)
```

Metric	Value
Tokens quota	640,375/100,000,000 used, 99,359,625 remaining
Resets in	6d 23h 35m 57s

Total consumed	640,375 tokens, \$0.348087 spent
Request tokens	27,997 in / 228 out (58 reasoning)

For the missing fico scores, we will replace them with the median of the same column. Give the count of missing ficos scores, the number of fico scores filled with median and the median value.

1 previous message

```
missing_fico_before = int(df["fico_score"].isna().sum())
fico_median = df["fico_score"].median()

df["fico_score"] = df["fico_score"].fillna(fico_median)

filled_with_median = missing_fico_before
missing_fico_after = int(df["fico_score"].isna().sum())

print("Missing fico_score (before):", missing_fico_before)
print("Median fico_score used:", fico_median)
print("FICO scores filled with median:", filled_with_median)
print("Missing fico_score (after):", missing_fico_after)
```

Metric	Value
Tokens quota	668,847/100,000,000 used, 99,331,153 remaining
Resets in	6d 23h 34m 40s
Total consumed	668,847 tokens, \$0.355581 spent
Request tokens	28,340 in / 132 out (0 reasoning)

We are done with the cleaning part. Now let us check what the shape of the data set is and the total missing values within the entire data set

1 previous message


```
print("Cleaned df shape:", df.shape)

total_missing = int(df.isna().sum().sum())
print("Total missing values in df:", total_missing)

# (Optional) missing by column, to confirm all are 0
df.isna().sum()
```

Metric	Value
Tokens quota	697,521/100,000,000 used, 99,302,479 remaining
Resets in	6d 23h 33m 58s
Total consumed	697,521 tokens, \$0.361765 spent
Request tokens	28,612 in / 62 out (0 reasoning)

Let's take a look at the business insight.

Let's see the count of approved loans and rejected loans. This gives us an understanding of the distribution. First provide with the count. Then convert the approved number to a percentage rate. After this, create a bar chart which gives the approved (In green) and rejected (In red) number. Please title it as Approved vs Rejected Loan Applications and also label both the axis. Don't forget to include the number on top of each bar.

1 previous message

```
import matplotlib.pyplot as plt

# Counts
decision_counts = df["approved"].value_counts()
print("Counts:\n", decision_counts)

# Approval percentage
approved_count = int(decision_counts.get("Approved", 0))
```

```

total_count = int(decision_counts.sum())
approved_rate = approved_count / total_count if total_c
print(f"\nApproval rate: {approved_rate:.2%}")

# Bar chart
labels = ["Approved", "Rejected"]
values = [int(decision_counts.get("Approved", 0)), int(
colors = ["green", "red"]

plt.figure(figsize=(6, 4))
bars = plt.bar(labels, values, color=colors)
plt.title("Approved vs Rejected Loan Applications")
plt.xlabel("Decision")
plt.ylabel("Number of Applications")

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, height,
             ha="center", va="bottom")

plt.tight_layout()
plt.show()

```

Metric	Value
Tokens quota	726,637/100,000,000 used, 99,273,363 remaining
Resets in	6d 23h 31m 17s
Total consumed	726,637 tokens, \$0.370603 spent
Request tokens	28,884 in / 232 out (0 reasoning)

There are 127 approved and 85 rejected. So that is 60% approval rate. Let's dig deeper into other insights. Can you help me build a histogram for FICO score with a vertical dashed line at 660 using the old flat rule cutoff. Please title the graph as "Distribution of FICO Scores" and title the axis. This will help us see which ones are lesser than the cutoff and how many are greater.

1 previous message

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(df["fico_score"], bins=20, edgecolor="black")
plt.axvline(660, color="red", linestyle="--", linewidth=2)

plt.title("Distribution of FICO Scores")
plt.xlabel("FICO Score")
plt.ylabel("Number of Applicants")
plt.legend()

plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	756,048/100,000,000 used, 99,243,952 remaining
Resets in	6d 23h 25m
Total consumed	756,048 tokens, \$0.377522 spent
Request tokens	29,312 in / 99 out (0 reasoning)

We have a good ditribution on either ends of the cutoff line. Let's see some other insights. Lets calculate the approval rate by employment_status to see if the approval depends on the employment type. Print the rates for each employment type for approvals. After that, build a bar chart that shows the same with percentages on top of the bars. Title the graph as "Approval Rate by Employment Type" and title the axis.

1 previous message

```
import matplotlib.pyplot as plt
```

```

# Approval rate by employment type (as %)
approval_rate_by_emp = (
    df.assign(approved_num=(df["approved"] == "Approved"
        .groupby("employment_status")["approved_num"]
        .mean()
        .sort_values(ascending=False)
        .mul(100)
    )

print("Approval rate by employment_status (%):")
print(approval_rate_by_emp.round(2))

# Bar chart
plt.figure(figsize=(8, 5))
bars = plt.bar(approval_rate_by_emp.index, approval_rate_by_emp.values)

plt.title("Approval Rate by Employment Type")
plt.xlabel("Employment Type")
plt.ylabel("Approval Rate (%)")
plt.xticks(rotation=30, ha="right")
plt.ylim(0, 100)

for bar in bars:
    h = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, h, f"{h}%")

plt.tight_layout()
plt.show()

```

Metric	Value
Tokens quota	785,907/100,000,000 used, 99,214,093 remaining
Resets in	6d 23h 19m 51s
Total consumed	785,907 tokens, \$0.386414 spent
Request tokens	29,630 in / 229 out (0 reasoning)

65% of salaried employees have approved loans vs 52%. This gives a slight idea on how employment type might matter. Now let's take a look at the fico score of approved applicants. This gives us an idea, if our cutoff method is reliable or not. Draw a seaborn box plot to look between approved and rejected loans fico scores and check their the overlap. Give the graph a title and label the axis.

1 previous message

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x="approved", y="fico_score")

plt.title("FICO Scores by Approval Decision")
plt.xlabel("Historical Decision")
plt.ylabel("FICO Score")

plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	816,061/100,000,000 used, 99,183,939 remaining
Resets in	6d 23h 14m 40s
Total consumed	816,061 tokens, \$0.393118 spent
Request tokens	30,080 in / 74 out (0 reasoning)

I can see that the boxes overlap although the approved applicants score is a bit higher. This tells us we cannot depend on our cutoff rule.

To understand the correlation between each columns, help me build a heatmap. Create a copy of df and add a temporary numeric column 1 for approved and 0 for rejected. Now using the numeric columns

which are `fico_score`, `annual_income`, `loan_amount`, `loan_term_months`, `years_employed`, `savings_balance`, `approved_num` build a seaborn heatmap of their correlations with numbers on each cell. Don't forget to title the graph. Please do not make any changes to the original df.

1 previous message

```
import matplotlib.pyplot as plt
import seaborn as sns

df_corr = df.copy()
df_corr["approved_num"] = (df_corr["approved"] == "Approved")

numeric_cols = [
    "fico_score",
    "annual_income",
    "loan_amount",
    "loan_term_months",
    "years_employed",
    "savings_balance",
    "approved_num",
]

corr = df_corr[numeric_cols].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap (Numeric Features + Approved)")
plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	846,653/100,000,000 used, 99,153,347 remaining
Resets in	6d 23h 4m 49s

Total consumed	846,653 tokens, \$0.401201 spent
Request tokens	30,435 in / 157 out (0 reasoning)

From the heatmap, you can say:

1. Loan amount has the strongest relationship with approval (-0.38), suggesting larger loan requests were more often linked with rejection in the historical data.
2. FICO score has a positive relationship with approval (0.26), so higher FICO scores were somewhat more associated with approvals.
3. Years employed also has a positive relationship with approval (0.23), suggesting longer employment history may have influenced past decisions.
4. Savings balance has a smaller positive relationship with approval (0.14), while annual income by itself has almost no direct relationship with approval (0.02).
5. No single factor has a very strong correlation with approval, which supports using several application factors together instead of deciding based only on FICO score.

Now, lets move on to model training.

Let's start with training one logistic regression model.

We currently have the loan decision stored as text in the `approved` column, with values approved and rejected . Since the model needs a numeric target, help me create a new column called `approved_flag` where approved is represented as 1 and rejected as 0. After creating the column, print the count of both 1 and 0 values so we can confirm that the conversion was done correctly

1 previous message

```
# Create numeric target: 1 = Approved, 0 = Rejected
df["approved_flag"] = df["approved"].astype(str).str.strip().map(
    {"approved": 1, "rejected": 0}
)
```

```
# Confirm conversion counts
print(df["approved_flag"].value_counts(dropna=False))
```

Metric	Value
Tokens quota	877,791/100,000,000 used, 99,122,209 remaining
Resets in	6d 22h 8m 35s
Total consumed	877,791 tokens, \$0.457542 spent
Request tokens	30,987 in / 151 out (72 reasoning)

We are now choosing the input features for the model and separating them from the target. Help me create X using only these columns: `fico_score`, `annual_income`, `loan_amount`, `loan_term_months`, `years_employed`, `savings_balance`. Also add a new column in X called `is_self_employed`, where the value is `1` if `employment_status` is Self-employed and `0` otherwise. Then create y using the `approved_flag` column. Please make sure that `applicant_id`, `approved`, and `approved_flag` are not included in X, since that would cause data leakage. Keep the variable names exactly as X and y because we will use them in the next steps. After creating them, print the shape of X so we can confirm the inputs were created correctly.

1 previous message

```
feature_cols = [
    "fico_score",
    "annual_income",
    "loan_amount",
    "loan_term_months",
    "years_employed",
    "savings_balance",
]

X = df[feature_cols].copy()
```



```
X["is_self-employed"] = (df["employment_status"] == "Self-employed")
y = df["approved_flag"].copy()

print("X shape:", X.shape)
```

Metric	Value
Tokens quota	909,265/100,000,000 used, 99,090,735 remaining
Resets in	6d 22h 4m 10s
Total consumed	909,265 tokens, \$0.464998 spent
Request tokens	31,379 in / 95 out (0 reasoning)

We can now split the data so we can train the model on one part and please test it honestly on data it has not seen before. Help me split $\mathbf{x}$ and $\mathbf{y}$ using scikit-learn's `train_test_split`. Use `test_size=0.2` so 80% of the data is used for training and 20% for testing, use `stratify=y` so the approval and rejection balance stays similar in both sets, and use `random_state=42` so we get the same split every time. Please name the outputs exactly `X_train`, `X_test`, `y_train`, and `y_test`. After the split, print how many rows are in the training set and how many are in the test set

1 previous message

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42,
)
```

```
print("Training rows:", X_train.shape[0])
print("Test rows:", X_test.shape[0])
```

Metric	Value
Tokens quota	941,116/100,000,000 used, 99,058,884 remaining
Resets in	6d 21h 16m 39s
Total consumed	941,116 tokens, \$0.472338 spent
Request tokens	31,770 in / 81 out (0 reasoning)

Now that we have split the data into training and test sets, help me train our logistic regression model. Help me create a logisticregression model using scikit-learn with max_iter=100 fit it using X_train and y_train, and name the model "model". Once the model is trained, print a short confirmation message to show that the training is complete.

1 previous message

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=100)
model.fit(X_train, y_train)

print("Logistic regression model training complete.")
```

Metric	Value
Tokens quota	973,225/100,000,000 used, 99,026,775 remaining
Resets in	6d 20h 49m 45s
Total consumed	973,225 tokens, \$0.479046 spent
Request tokens	32,067 in / 42 out (0 reasoning)

Now that we have split the data into training and test sets, let's train our logistic regression model. Help me create a Logistic Regression model using scikit-learn with max_iter=1000, fit it using X_train and y_train, and name the model "Model". Once the model is trained, print a short confirmation message to show that the training is complete.

1 previous message

```
from sklearn.linear_model import LogisticRegression

Model = LogisticRegression(max_iter=1000)
Model.fit(X_train, y_train)

print("Logistic Regression model (Model) training complete")
```

Metric	Value
Tokens quota	1,005,596/100,000,000 used, 98,994,404 remaining
Resets in	6d 20h 36m 48s
Total consumed	1,005,596 tokens, \$0.536259 spent
Request tokens	32,325 in / 46 out (0 reasoning)

Now that our logistic regression model is trained, let's understand what it has learned from each feature. Help me create a table that pairs each feature name from X.columns with its coefficient from model.coef_[0] and sort the values from lowest to highest. Then create a horizontal bar chart titled "What the Model Learned (Feature Weights)", using green bars for positive coefficients and red bars for negative coefficients. This will help us see which factors push the model more toward approval and which push it more toward rejection.

1 previous message

```
import pandas as pd
import matplotlib.pyplot as plt

# Coefficient table
coef_df = pd.DataFrame({
    "feature": X.columns,
    "coefficient": model.coef_[0],
}).sort_values("coefficient", ascending=True)

coef_df
```

```
# Horizontal bar chart (green = positive, red = negative)
colors = ["green" if c > 0 else "red" for c in coef_df["coefficient"]]

plt.figure(figsize=(8, 5))
plt.barh(coef_df["feature"], coef_df["coefficient"], color=colors)
plt.title("What the Model Learned (Feature Weights)")
plt.xlabel("Coefficient (Weight)")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	1,038,375/100,000,000 used, 98,961,625 remaining
Resets in	6d 20h 24m 51s
Total consumed	1,038,375 tokens, \$0.544816 spent
Request tokens	32,616 in / 163 out (0 reasoning)

Now that we have looked at the raw feature weights, I want to compare the features fairly because they are measured on very different scales. Help me import `StandardScaler` from `sklearn.preprocessing`, fit it only on `X_train`, and create a scaled copy of the training data without changing the original `X_train` or `X_test`.

Then train a new and separate LogisticRegression(max_iter=1000) model called importance_model using the scaled training data and y_train. After that, pair each feature name from X_train.columns with the coefficients from importance_model.coef_[0], sort them, and create a horizontal bar chart titled "Which Factors Matter Most (Fairly Compared)", using green for positive coefficients and red for negative coefficients. Please make sure the original model, X_train, and X_test are not changed

1 previous message

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
import pandas as pd
import matplotlib.pyplot as plt

# Fit scaler ONLY on X_train (do not overwrite X_train)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # created for

# Train a separate model on the SCALED training data
importance_model = LogisticRegression(max_iter=1000)
importance_model.fit(X_train_scaled, y_train)

# Coefficient table (scaled => more comparable)
importance_df = pd.DataFrame({
    "feature": X_train.columns,
    "coefficient": importance_model.coef_[0],
}).sort_values("coefficient", ascending=True)

importance_df

# Horizontal bar chart
colors = ["green" if c > 0 else "red" for c in importance_model.coef_[0]]

plt.figure(figsize=(8, 5))
plt.barh(importance_df["feature"], importance_df["coefficient"], color=colors)
```

```
plt.title("Which Factors Matter Most (Fairly Compared) ")
plt.xlabel("Coefficient (Scaled Feature Weight) ")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	1,071,973/100,000,000 used, 98,928,027 remaining
Resets in	6d 6h 51m 1s
Total consumed	1,071,973 tokens, \$0.606846 spent
Request tokens	33,334 in / 264 out (0 reasoning)

The model is trained, so let's test how accurately it predicts the historical decisions on the test data. Help me use the trained model to make predictions on `X_test`, compare those predictions with `y_test` using `accuracy_score`, and store the result in a variable called `accuracy`. Then print the accuracy as a percentage with one decimal place so we can clearly see how well the model performed.

1 previous message

```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Test accuracy: {accuracy * 100:.1f}%")
```

Metric	Value
--------	-------

Tokens quota	1,105,841/100,000,000 used, 98,894,159 remaining
Resets in	6d 6h 44m 1s
Total consumed	1,105,841 tokens, \$0.613896 spent
Request tokens	33,819 in / 49 out (0 reasoning)

Let us try and see what the accuracy for our old flat rule is and compare it with the logistic regression model. Help me create predictions on X_test where an application is approved as 1 if fico_score is greater than or equal to 660, and rejected as 0 otherwise. Then compare these predictions with y_test using accuracy_score, store the result in a variable called flat_rule_accuracy, and print the accuracy as a percentage with one decimal place

1 previous message

```
from sklearn.metrics import accuracy_score

flat_rule_pred = (X_test["fico_score"] >= 660).astype(int)

flat_rule_accuracy = accuracy_score(y_test, flat_rule_pred)

print(f"Flat rule test accuracy: {flat_rule_accuracy * 100:.1f}%")
```

Metric	Value
Tokens quota	1,140,017/100,000,000 used, 98,859,983 remaining
Resets in	6d 6h 37m 5s
Total consumed	1,140,017 tokens, \$0.621290 spent

Request tokens

34,110 in / 66 out (0 reasoning)

Now that we have the accuracy of our logistic regression model and the old FICO flat rule, let's also compare them with a simple baseline where every application is predicted as Approved. Help me calculate the accuracy of always predicting 1 for every row in `y_test`. Then create a bar chart comparing the model accuracy using `accuracy`, the old flat rule using `flat_rule_accuracy`, and the always-approve baseline. Show all three as percentages, title the chart "Model vs Flat Rule vs Baseline", and display the accuracy value on top of each bar

1 previous message

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score

# Always-approve baseline (predict 1 for every test row)
baseline_pred = np.ones_like(y_test)
baseline_accuracy = accuracy_score(y_test, baseline_pred)

# Bar chart comparison (percent)
labels = ["Logistic Model", "Flat Rule (>=660)", "Always-approve"]
values = [accuracy * 100, flat_rule_accuracy * 100, baseline_accuracy * 100]

plt.figure(figsize=(8, 5))
bars = plt.bar(labels, values, color=["steelblue", "orange", "lightcoral"])
plt.title("Model vs Flat Rule vs Baseline")
plt.xlabel("Approach")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)

for bar in bars:
    h = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, h, f"{h}%")

plt.tight_layout()
plt.show()
```


Metric	Value
Tokens quota	1,174,668/100,000,000 used, 98,825,332 remaining
Resets in	6d 6h 31m 39s
Total consumed	1,174,668 tokens, \$0.630999 spent
Request tokens	34,431 in / 220 out (0 reasoning)

Now that we know the model's accuracy, let's see how many applications it could confidently sort without needing a human review. Help me use `predict_proba` on `X_test` to get the probability of class 1, which is approval. Then sort each application into three groups: "Auto-approve" if the probability is above 0.80, "Auto-reject" if it is below 0.20, and "Send to human" for everything in between. After that, print how many test applications fall into each group and create a bar chart titled "How the Confidence Rule Sorts Applications" showing the count in each bucket

1 previous message

```
import pandas as pd
import matplotlib.pyplot as plt

# Probabilities of approval (class 1)
proba_approve = model.predict_proba(X_test)[: , 1]

# Bucket by business thresholds
buckets = pd.Series(proba_approve).apply(
    lambda p: "Auto-approve" if p > 0.80 else ("Auto-re
)

bucket_counts = buckets.value_counts().reindex(["Auto-a
print(bucket_counts)

# Bar chart
```

```
plt.figure(figsize=(7, 4))
bars = plt.bar(bucket_counts.index, bucket_counts.value)
plt.title("How the Confidence Rule Sorts Applications")
plt.xlabel("Bucket")
plt.ylabel("Number of Test Applications")

for bar in bars:
    h = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, h, str(h))

plt.tight_layout()
plt.show()
```

Metric	Value
Tokens quota	1,209,831/100,000,000 used, 98,790,169 remaining
Resets in	6d 6h 23m 25s
Total consumed	1,209,831 tokens, \$0.641042 spent
Request tokens	34,923 in / 240 out (0 reasoning)

Great, everything looks good. Can you now generate a README.md for the project, summarizing what the notebook does, its four broad steps, providing a summary of project

3 previous messages

Metric	Value
Tokens quota	1,246,848/100,000,000 used, 98,753,152 remaining
Resets in	6d 5h 25m 16s
Total consumed	1,246,848 tokens, \$0.726720 spent

Request tokens	35,311 in / 1,706 out (317 reasoning)
-----------------------	---------------------------------------

Created a file

- Wrote `./README.md` (+109 -0)

Added `README.md` with:

- Business problem + objective (historical approval prediction, not default risk)
- Dataset description
- Four broad project stages
- Setup/dependencies + how to run
`Learner_Guide_Notebook.ipynb`
- Cleaning summary (money parsing, duplicates, impossible values rules, median FICO fill)
- Modeling approach (features, leakage exclusions, split settings, logistic regression, scaled coefficient comparison)
- Evaluation section (leaves key numeric results as “see notebook output” to avoid inventing numbers)
- Fairness/risk notes, limitations, and appropriate use as decision-support

Metric	Value
Tokens quota	1,284,139/100,000,000 used, 98,715,861 remaining
Resets in	6d 5h 25m 4s
Total consumed	1,284,139 tokens, \$0.738120 spent
Request tokens	37,162 in / 129 out (0 reasoning)